

POCKET SMART AI — Your Smart Budget & Recommendation Assistant

Phase 5: Project Development

St. George's Arts and Science College (UNM1727) · TN Skills | Generative AI · Team ID: SWTID-2026-2087

Team Leader: Monish B | Members: Jagadeeshwaran S, Shannon Cadric Jacob, Preethika B, Naresh S

5.1 Purpose of This Phase

This phase covers the actual build: environment configuration, the core features implemented, and the integration/optimization work that turned the individual pieces into one working application.

5.2 Environment Setup & Configuration

- Installed the project runtime, package manager, and IDE tooling, and verified the application starts locally.
- Installed dependencies from requirements.txt and confirmed all packages resolved without missing-module errors.
- Created the working project structure: templates/, static/, tests/, app.py, database.py, gemini_utils.py, .env / .env.example, requirements.txt, README.md, pytest.ini.
- Kept environment-specific values (local ports, credentials, database connection values, API keys) outside source code using environment configuration rather than hard-coded secrets.

5.3 Feature 1 — User Access

A registration/login-oriented screen flow (Create Account and Login pages) establishes the entry point and basic user journey. Successful login lands the user on a personalized dashboard ('Hello, <name>').

5.4 Feature 2 — Budget & Requirement Processing

The core feature is fixed-budget planning across multiple interdependent categories. The user selects a planner (Home Interior, Party, or Jewelry), enters the total budget and domain-specific requirements (e.g. rooms, furniture counts, guest counts, occasion, style preferences, and an optional outfit image for the Jewelry planner), and the application checks the total budget constraint before preparing an allocation-oriented result.

5.5 Feature 3 — Recommendation / Dashboard Output

Processed information is presented in a dashboard-oriented result page showing the total budget, remaining amount, allocated amount, and a per-category breakdown with item-level recommendations and store-link tags (e.g. Amazon, Flipkart). A saved-history view lists prior plans with their remaining balance and timestamp.

5.6 Integration & Optimization (Milestone 4)

Component Integration

The user interface was integrated with the application's processing logic so that inputs move through validation, calculation/recommendation, and result rendering as one continuous flow.

Performance Optimization

- Validate input early to avoid wasted processing.
- Avoid repeated/duplicate processing of the same request.
- Minimize unnecessary network requests.

- Keep result generation as deterministic as practical.
- Optimize static assets for faster loading.

Security Enhancements

- Validate all user input on submission.
- Protect authentication data and avoid exposing secrets in source code.
- Use HTTPS in production.
- Apply least-privilege access to persistent storage and external services.

Error Handling & Validation

The application handles missing fields, invalid numeric values, impossible budget combinations, service/API failures, and unexpected errors with user-friendly messages and safe fallbacks, rather than crashing.

5.7 Database / API / Model Notes

Persistent plan history is stored locally via SQLite (as shown in the application's saved-recommendations screen), and the recommendation layer is powered by Google Gemini 1.5 Flash Pro, invoked through `gemini_utils.py`. Any further schema, endpoint, or model configuration detail should be verified against the project repository as the authoritative source.

5.8 Outcome of This Phase

A working application implementing user access, budget/requirement input, AI-assisted allocation and recommendation output, and history tracking — integrated into one continuous, validated flow and ready for structured testing.